MODULE 16 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Connectors, integrations, and the Domain 3 drill

The last pieces of the ingestion surface, then five scenarios across modules 13 to 16.

DOMAIN 3 of 5 WEIGHT 18% MODULE 16 of 31

admin@sthithapragna.com

The connector landscape

CONNECTOR	WHAT IT IS FOR
Kafka connector	Streams Kafka topics into tables. Two ingestion methods, covered next.
Spark connector	Reads and writes Snowflake from an existing Spark cluster.
JDBC and ODBC	Native drivers. Third-party BI and ETL tools, Java and C applications.
Python, Node.js, Go, .NET, PHP	Programmatic interfaces for application code.
SQL API	A REST API. No driver to install, so it suits serverless functions.

ON THE EXAM

All Standard Edition. The choice is about the workload, never the edition.

The Kafka connector has two modes

- **snowflake.ingestion.method** is either **SNOWPIPE**, which is file-based, or **SNOWPIPE_STREAMING**, which writes rows.
- By default the target table has just **two VARIANT columns: RECORD_CONTENT and RECORD_METADATA**.
- **Schema detection and evolution works only with SNOWPIPE_STREAMING.** It is not supported with file-based Snowpipe.
- Enable it with **snowflake.enable.schematization**, plus **ENABLE_SCHEMA_EVOLUTION** on the table, which needs OWNERSHIP.

> ON THE EXAM
>
> The same files-versus-rows split as module 15, surfacing again. Only the row-based mode can shape a table to the data.

Reaching outward from Snowflake

- An **external function** calls code hosted outside Snowflake, through a proxy such as API Gateway. It needs an **API integration**.
- External functions are **not** the same as external tables. One calls a remote service; the other reads remote files.
- **External network access** lets a UDF or stored procedure reach an external endpoint, using a **network rule** with MODE = EGRESS.
- On Business Critical, Amazon API Gateway **private endpoints** are supported for external functions.

> ON THE EXAM
>
> Note the network rule reappearing here in EGRESS mode. Same object as module 9, pointed the other way.

The four integration types, finally side by side

INTEGRATION	WHAT IT CONNECTS TO
Storage	Cloud object storage. S3, Azure Blob, GCS. Used by external stages.
API	An HTTPS endpoint. Used by external functions and by Git repositories.
Notification	Cloud messaging. Used by Snowpipe auto-ingest and by error notifications.
Catalog	An external Iceberg catalog such as AWS Glue or a REST catalog.

ON THE EXAM

Four types, four purposes, never interchangeable. Picking the wrong one is the commonest integration error on this exam.

> A team configures the Kafka connector with snowflake.ingestion.method set to SNOWPIPE, and sets snowflake.enable.schematization to TRUE, expecting their JSON messages to arrive as proper typed columns.
>
> Data arrives correctly but every table has only two columns, both VARIANT, named RECORD_CONTENT and RECORD_METADATA.

Why is schematization not working?

A. ENABLE_SCHEMA_EVOLUTION defaults to TRUE and must be explicitly disabled before schematization takes effect.

B. Schema detection and evolution works only with SNOWPIPE_STREAMING, not with file-based Snowpipe.

C. The connector needs a storage integration before it can create columns in a target table.

D. JSON cannot be schematized. Only Avro messages carry enough type information.

◆ PAUSE AND DECIDE

Why is schematization not working?

A ENABLE_SCHEMA_EVOLUTION must be disabled.

Backwards, and it defaults to FALSE. It must be set to TRUE, by a role with OWNERSHIP on the table.

B It works only with SNOWPIPE_STREAMING.

Documented explicitly. The file-based Snowpipe method does not support schema detection and evolution at all.

C A storage integration is needed.

Storage integrations serve external stages. Nothing here is reading cloud object storage.

D Only Avro can be schematized.

JSON schematizes perfectly well. The ingestion method is the constraint, not the message format.

WHERE THIS CAME FROM

Module 16, following module 15. The files-versus-rows split decides more than latency: only the row-based path can shape a table.

> A daily COPY loads a partner's CSV export into a wide table. For six weeks it works. Then the partner adds a new column in the middle of their export without telling anybody.
>
> The COPY continues to succeed every night with no errors. Two weeks later an analyst notices that postcodes are appearing in the city column and dates are shifted by one field.

What went wrong, and what prevents it?

A The file format's delimiter changed. Set MATCH_BY_COLUMN_NAME to detect delimiter drift.

B Load metadata expired after 64 days, so files were reloaded against a changed schema.

C ON_ERROR was set to CONTINUE, which silently skipped the malformed rows.

D COPY maps columns by position by default. Use MATCH_BY_COLUMN_NAME to map by name instead.

◆ PAUSE AND DECIDE

What went wrong, and what prevents it?

A Delimiter drift.

A changed delimiter would produce parse errors, not neatly shifted data. MATCH_BY_COLUMN_NAME does not detect delimiters.

B 64-day metadata expiry.

That governs whether a file is reloaded. It cannot move values between columns.

C ON_ERROR = CONTINUE hid it.

CONTINUE skips rows that error. These rows did not error; they loaded successfully into the wrong columns.

D Positional mapping. Use MATCH_BY_COLUMN_NAME.

Default mapping is by position, so an inserted column shifts everything after it. The load succeeds, which is what makes it dangerous.

WHERE THIS CAME FROM

Module 14. This is the failure mode that produces no error at all, which is why it ran for two weeks before anyone noticed.

> An IoT platform writes one small JSON file per device per minute to an external stage. Around 40,000 files arrive each hour, averaging 15 KB each. Snowpipe auto-ingest loads them.
>
> The data lands correctly and on time, but the Snowpipe line on the bill is far larger than the volume of data suggests it should be.

What is the cause, and what should change?

A. Snowpipe has a per-file overhead charge. Aggregate into fewer, larger files, or use Snowpipe Streaming.

B. Snowpipe is billing warehouse credits. Assign a smaller warehouse to the pipe.

C. Auto-ingest is more expensive than the REST API. Switch the platform to calling the REST API.

D. The pipe needs PURGE = TRUE, because retained staged files are being re-scanned and re-billed.

◆ PAUSE AND DECIDE

What is the cause, and what should change?

A Per-file overhead. Aggregate, or use Streaming.

40,000 files an hour at 15 KB each is the pattern the sizing guidance warns against. Overhead dominates the actual data cost.

B Assign a smaller warehouse.

Snowpipe is serverless. You never assign it a warehouse, so there is nothing to resize.

C The REST API is cheaper.

Both trigger the same loading work. The trigger mechanism is not where the cost is.

D Set PURGE = TRUE.

Snowpipe does not support PURGE at all. Loaded files are also not re-scanned.

WHERE THIS CAME FROM

Modules 13 and 15. The mirror of the one-huge-file scenario: too many tiny files rather than too few large ones.

A pipeline uses an append-only stream on a source table, consumed hourly by a task that inserts new rows into a reporting table. It has run reliably for months.

Auditors find that customer records deleted from the source table are still present in the reporting table, and that corrections applied as updates never reached it.

What is the cause?

A The stream went stale, so deletes and updates were lost when the offset was reset.

B An append-only stream tracks inserts only. Updates and deletes are never recorded.

C The task ran with insufficient privileges to apply deletes, and failed silently.

D Deletes are only visible in a stream if the source table has Time Travel of at least 7 days.

◆ PAUSE AND DECIDE

What is the cause?

A The stream went stale.

Staleness loses everything unconsumed, not selectively deletes and updates. The pipeline also ran reliably throughout.

B Append-only tracks inserts only.

Documented. Updates, deletes and truncates are simply not recorded. A standard stream is needed to see them.

C Insufficient privileges.

The task only inserts, and it never attempted a delete. There was nothing to be denied.

D Deletes need 7 days of Time Travel.

Invented. Retention governs how long a stream stays valid, not which operations appear in it.

WHERE THIS CAME FROM

Module 15. The pipeline never broke. It was doing precisely what an append-only stream and an insert-only task were built to do.

> A team wants a Snowflake stored procedure to call an internal pricing service over HTTPS, and separately wants Snowflake to read Apache Iceberg tables from a data lake whose catalog is AWS Glue.
>
> An engineer proposes creating one storage integration and using it for both, since both reach outside Snowflake.

What is actually required?

A. One API integration covers both, since Glue is reached over an HTTPS API.

B. A notification integration for the pricing service, and a storage integration for the Iceberg tables.

C. External network access with an EGRESS network rule for the pricing service, and a catalog integration for Glue.

D. The storage integration works for both, provided its allowed locations list includes the pricing service URL.

◆ PAUSE AND DECIDE

What is actually required?

A One API integration for both.

API integrations do serve HTTPS endpoints, and an external Iceberg catalog needs a catalog integration specifically.

B Notification plus storage.

Notification integrations carry cloud messaging events. Neither requirement involves messaging.

C External network access, plus a catalog integration.

A procedure reaching an endpoint uses external network access with an EGRESS network rule. An external Iceberg catalog needs a catalog integration.

D Storage integration with an added URL.

The engineer's proposal. Storage integrations address object storage locations, not service endpoints.

WHERE THIS CAME FROM

Modules 4, 9, 13 and 16 together. Two requirements that both 'reach outside Snowflake' and need entirely different objects.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Kafka schematization did nothing	Module 16 - the Kafka connector's two modes
2. Columns silently shifted	Module 14 - MATCH_BY_COLUMN_NAME
3. Snowpipe bill far larger than the data	Modules 13 and 15 - file sizing and per-file overhead
4. Deletes never reached the reporting table	Module 15 - the three stream types
5. Two ways of reaching outside Snowflake	Modules 4, 9, 13 and 16 - the four integration types

ON THE EXAM

Questions 3 and 5 span several modules. Those are the ones worth re-reading even if you got them right.

Domain 3 in five sentences

- **Loading is two steps.** Files reach a stage, then COPY INTO makes them rows. Snowpipe automates step two.
- **File count decides parallelism.** Too few files wastes the warehouse; too many wastes money on overhead. Aim for 100 to 250 MB.
- **Defaults nobody chose:** ABORT_STATEMENT for bulk, SKIP_FILE for Snowpipe, positional column matching, MAX_FILE_SIZE 16 MB.
- **Files or rows** separates Snowpipe from Snowpipe Streaming, and decides Kafka schematization too.
- **Four integration types**, never interchangeable. Storage, API, notification, catalog.

> ON THE EXAM
>
> Next: Domain 4. Performance, querying and transformation at 21%, beginning with warehouse sizing and scaling.